

CSA15 Cloud Computing and Big Data Analytics

Capstone Cloud Technical Presentation

Individual Student Presentation Deck

Presentation Focus

Present your capstone as a cloud-backed product. Explain the problem, architecture, service choices, evidence, and CO1 learning clearly.

Presenter Details

Student Name: Karanam Reddy Vishnu

Chaithanya

Register Number: 192372057

Class / Section / Slot: B

Capstone Title: SentinelX – Cloud Threat Intelligence (NLP + Serverless Risk Scoring)

Selected SDG Goal(s): SDG 9 – Industry, Innovation and Infrastructure

Cloud Platform / Tools: AWS Lambda, MongoDB Atlas, React, Node.js

Problem Context and Stakeholder Mapping

Our capstone begins with a real problem to solve

Problem Statement

Security teams are flooded with unstructured threat logs and alerts, making it hard to spot and respond to real attacks in time.

Target Users

SOC analysts, IT security teams, and small organizations that lack dedicated round-the-clock security staff.

SDG Fit

SDG 9 – Industry, Innovation and Infrastructure: builds resilient, secure digital infrastructure for organizations.

Product Boundary

V1 ingests threat logs, runs NLP-based analysis, computes a serverless risk score, and shows alerts on a dashboard.

Cloud Adoption Rationale and Concept Map

The problem needs cloud support to become a usable product

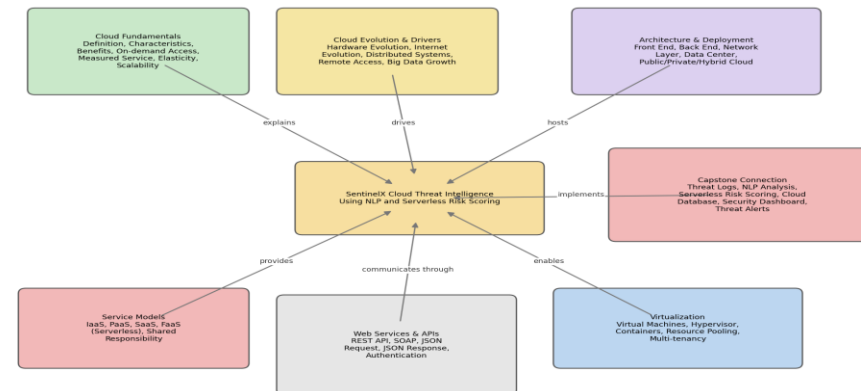
Cloud Definition

For SentinelIX, the cloud delivers on-demand compute, NLP processing, and storage over the internet so threat analysis scales without fixed servers.

Cloud Characteristics

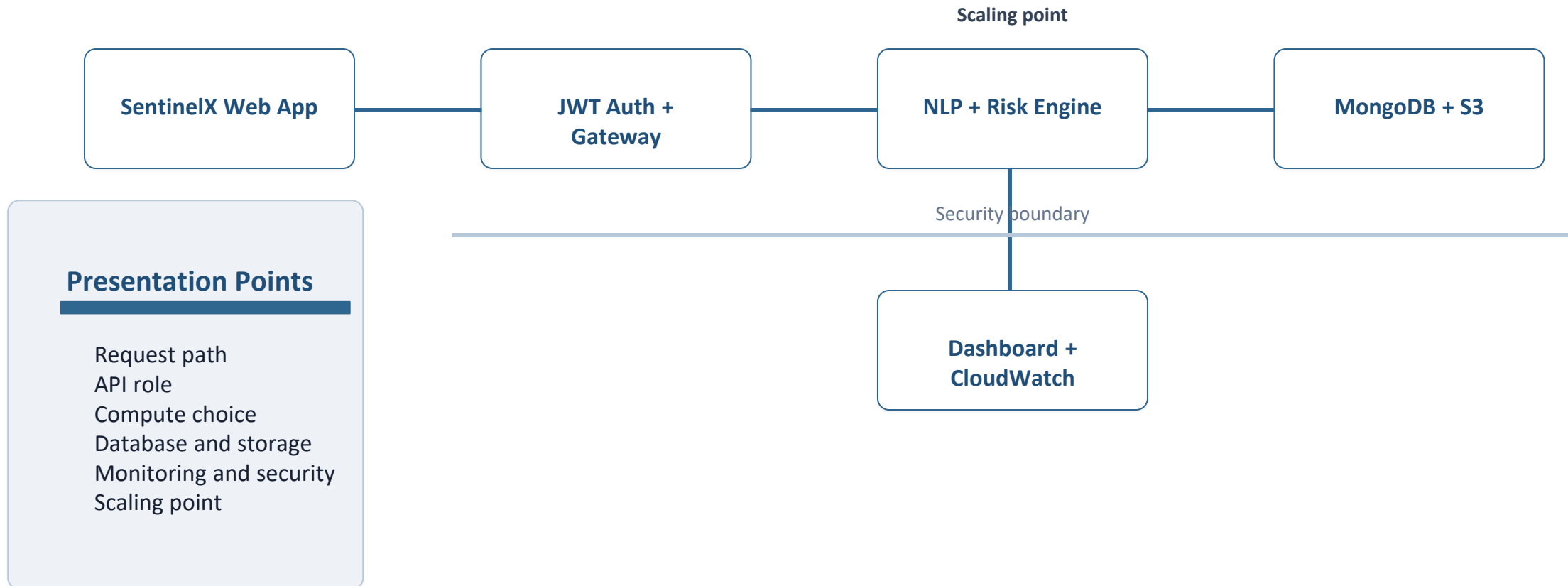
Elasticity auto-scales risk-scoring functions; scalability absorbs log spikes; measured service bills per invocation; resource pooling shares infrastructure.

Cloud Concept Map



Capstone Cloud Architecture

Our cloud architecture connects the user action to services



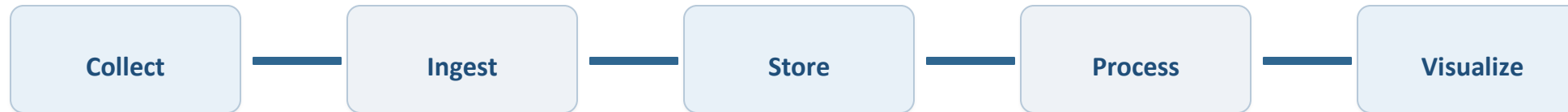
Cloud Service Model Responsibility Matrix

Each service model defines what we control and what cloud manages

Service area	Model	Provider manages	Student team manages	Why this fits
Frontend hosting	PaaS Hosting	Hosting, CDN, SSL, scaling	React UI, routing, dashboard UX	Fast delivery of the security dashboard
Backend/API	PaaS + FaaS	Runtime, patching, auto-scaling	REST APIs, NLP logic, risk scoring	Runs threat analysis and scores risk on demand
Database	SaaS (DBaaS)	Replication, backups, patching	Schema design, queries, indexes	MongoDB Atlas stores logs and risk scores
Storage	IaaS Object Storage	Durability, storage infra	Bucket structure, access rules	AWS S3 stores log files and reports
Monitoring / logs	SaaS (Managed)	Metric collection, alert infra	Dashboards, alert thresholds	CloudWatch tracks server health and usage
Security / IAM	SaaS / Managed IAM	Token infra, encryption at rest	JWT logic, roles and permissions	Firebase Auth / JWT secures API access

Data Flow and Analytics Pipeline

The product data moves from user activity to useful insight



Data Requirements

Threat log files, security alerts, and network/user activity events uploaded by analysts.

Analytics Requirement

Risk-score dashboard, real-time threat alerts, and trend reports for SOC analysts.

Proof Required

Sample threat log in, NLP-classified risk score out, and one detected high-risk alert on the dashboard.

Python / AI Module Integration

Python or AI adds intelligence where the product needs it

Python Role

NLP-based threat classification and automated risk scoring for incoming log data.

Input and Output

Input: raw threat log text. Output: risk category, confidence score, and severity level.

Execution Point

Deployed as a serverless function (AWS Lambda / FaaS) triggered by each incoming threat log.

Product Value

Cuts manual triage time and helps analysts prioritize the highest-risk threats first.

Security, Scalability, Privacy and Cost Design

Security, scale, privacy and cost shape the final design

Security

JWT-based authentication, role-based access, input validation, and an audit trail for every request.

Privacy

Threat log data is access-controlled and anonymized where possible; only authorized analysts view raw logs.

Scalability

Serverless risk-scoring functions auto-scale with log volume, so growth needs no manual redesign.

Cost Awareness

Uses free-tier cloud services and managed database tiers, and monitors API/storage usage to control cost.

Implementation Stack and Deployment Evidence

Our implementation stack shows how the design will be built

Mobile / Web Layer

React.js web application powering the SentinelX security dashboard.

Cloud Services

Node.js/Express API, MongoDB Atlas, AWS Lambda, AWS S3, CloudWatch, Firebase/JWT authentication.

Evidence

GitHub repository, deployed API endpoint, cloud dashboard screenshots, and CloudWatch logs.

Version Control

Organized repository with feature branches, clear commit history, and a documented README.

Demo Storyboard and User Journey

A complete user journey proves the product flow

Scene 1

Analyst logs in -> JWT authenticates the session -> **Firestore Auth / API Gateway**

Scene 2

Analyst uploads a threat log -> file reaches the API -> **Node.js + Express backend**

Scene 3

Backend sends log text to NLP -> threat patterns are extracted -> **NLP Threat Analysis module**

Scene 4

Serverless function scores risk -> risk level is calculated -> **AWS Lambda (FaaS)**

Scene 5

Result is saved -> risk score & log are stored -> **MongoDB Atlas**

Scene 6

Dashboard updates -> analyst sees alert & score -> **Security Dashboard + CloudWatch**

Evaluation Readiness Checklist

Our evidence confirms that the presentation is ready

☐

Cloud ecosystem and service model explained

☐

Elasticity and scalability decisions are visible

☐

REST/SOAP/API role connected to capstone

☐

GitHub and cloud evidence are inspectable

☐

Architecture has compute, data, storage, security, monitoring

☐

References and integrity statement are included

Evaluator Focus

CO1 is verified through cloud design reasoning, service choices, evidence, and individual explanation.

Conclusion, Limitations and Future Scope

We close with learning, limitations and next steps

Learning Outcome

Gained hands-on understanding of service models, elasticity, serverless computing, and API-driven cloud architecture.

Limitations

Real-time streaming ingestion and multi-language threat log support are not yet implemented.

Future Improvement

Add real-time log streaming, broaden language coverage, and automate incident-response actions.

Integrity Statement

Built using AWS, MongoDB Atlas, React, and Node.js documentation; AI assistance was used for planning and drafting, and is acknowledged here.